



PDF Download
3731569.3764840.pdf
29 March 2026
Total Citations: 0
Total Downloads: 3743



Published: 12 October 2025

Citation in BibTeX format

SOSP '25: ACM SIGOPS 31st Symposium
on Operating Systems Principles
October 13 - 16, 2025
Seoul, Republic of Korea

Conference Sponsors:
SIGOPS

DL Latest updates: <https://dl.acm.org/doi/10.1145/3731569.3764840>

RESEARCH-ARTICLE

Tempo: Compiled Dynamic Deep Learning with Symbolic Dependence Graphs

PEDRO F. SILVESTRE, Imperial College London, London, U.K.

PETER PIETZUCH, Imperial College London, London, U.K.

Open Access Support provided by:
Imperial College London



TEMPO: Compiled Dynamic Deep Learning with Symbolic Dependence Graphs

Pedro F. Silvestre
p.silvestre21@imperial.ac.uk
Imperial College London

Peter Pietzuch
prp@imperial.ac.uk
Imperial College London

Abstract

Deep learning (DL) algorithms are often defined in terms of *temporal relationships*: a tensor at one timestep may depend on tensors from earlier or later timesteps. Such *dynamic* dependencies (and corresponding dynamic tensor shapes) are difficult to express and optimize: while *eager* DL systems support such dynamism, they cannot apply compiler-based optimizations; *graph-based* systems require static tensor shapes, which forces users to pad tensors or break-up programs into multiple static graphs.

We describe TEMPO, a new DL system that combines the dynamism of eager execution with the whole-program optimizations of graph-based compilation. TEMPO achieves this through a declarative programming model with *recurrent tensors*, which include explicit *temporal dimensions*. Temporal dimensions can be indexed using *symbolic expressions* to express dynamic dependencies on past and future tensors. Based on this, TEMPO constructs a *symbolic dependence graph*, which concisely encodes dynamic dependencies between operators, and applies whole-program optimizations, such as algebraic simplifications, vectorization, tiling, and fusion. By tiling dynamic dependencies into static-size blocks, TEMPO can also reuse existing static code-generators. It then uses a polyhedral model to find a feasible execution schedule, which includes memory management operations. We show that TEMPO achieves a 7× speedup over JAX for Llama-3.2-3B decoding; for reinforcement learning algorithms, TEMPO achieves a 54× speedup, with 16× lower peak memory usage.

CCS Concepts: • **Computing methodologies** → *Machine learning*; *Symbolic and algebraic manipulation*.

Keywords: Deep Learning, Compilation, Dynamic Tensors

ACM Reference Format:

Pedro F. Silvestre and Peter Pietzuch. 2025. TEMPO: Compiled Dynamic Deep Learning with Symbolic Dependence Graphs. In *ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*, October 13–16, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3731569.3764840>



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1870-0/2025/10

<https://doi.org/10.1145/3731569.3764840>

1 Introduction

Deep Learning (DL) has had enormous success in domains such as robotics [1], games [2], natural language [3], protein folding [4], and image generation [5]. DL computation is defined using operators (e.g., matrix multiplication, addition, and indexing) with input and output *tensors*. Tensors are multi-dimensional arrays laid out spatially in memory, whose *shape*—a tuple of integers—specifies the size of each spatial dimension. DL workloads are resource-intensive, and DL systems, such as Torch [6] or JAX [7], offer high-level tensor programming models for GPU accelerators [8].

A common feature of many DL algorithms [5, 9–12] is their mathematical definition through *recurrence equations* [13]. Recurrence equations express *temporal* relationships between values at different timesteps (past and future) declaratively, i.e., without specifying an order of execution. For example, attention [9]—a key operation in large language models (LLMs)—depends, at a given timestep, on outputs from all prior timesteps; reinforcement learning (RL) [14, 15] algorithms produce actions at each timestep and use a sum of future rewards to evaluate them.

Such *dynamic* (i.e., time-varying) dependencies between operators and tensors of different timesteps introduce challenges when existing DL systems try to support them:

- **Expressiveness**—dynamic dependencies can create tensors with *dynamic shapes* whose support is limited in terms of current APIs [16, 17];
- **Optimization**—existing graph representations [7, 18] cannot express dynamic dependencies, missing opportunities for algebraic simplification or vectorization;
- **Scheduling**—future dependencies complicate scheduling, as dependent computations must be delayed [13, 19];
- **Memory management**—dependencies on past timesteps result in complex decisions on which tensors to retain, deallocate or evict with limited GPU memory [20].

Eager DL systems, such as Torch [6] or TF-Eager [21], support dynamism through the host language (Python) by dispatching GPU kernels on-demand [22]. This imperative approach, however, prevents them from performing compiler-based optimizations [16] or scheduling; in contrast, *graph-based* systems, such as TensorFlow [18] and JAX [7], enable ahead-of-time compilation by first constructing a *dataflow graph* in which each node represents a DL operator and edges are tensors that flow between them. Optimizations such as

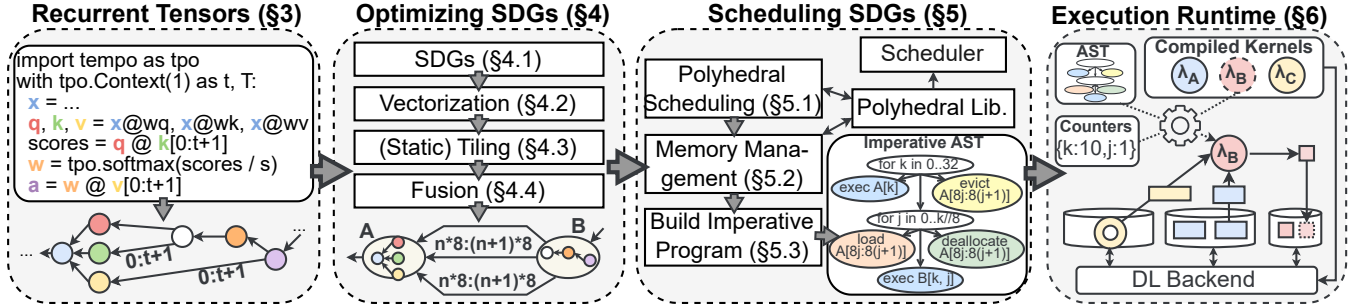


Fig. 1: Overview of how TEMPO expresses, optimizes, schedules and manages the memory of dynamic DL algorithms

memory planning [23, 24], operator fusion [25], vectorization [26], and code-generation [27, 28], however, require statically known fixed-size tensor shapes.

To compile dynamic DL algorithms, users must *pad* dynamic tensor shapes to a static size, and use *masks* to encode dynamic dependencies, which leads to large memory and execution overheads [29]. Alternatively, users can break-up computation into static sub-programs, but this prevents whole-program optimizations leading to worse performance [30].

Our key insight is that tensors in dynamic DL algorithms have implicit *temporal dimensions*. By making them explicit, dynamic dependencies become regular tensor-indexing operations across temporal dimensions. Furthermore, we show that dynamism in dependencies and shapes has structure, and can be concisely represented using *symbolic expressions* of timestep symbols, enabling optimization.

We describe TEMPO, a new DL system that expresses, optimizes, and schedules dynamic DL programs, as well as manages their memory. TEMPO makes the following novel technical contributions (shown in Fig. 1):

(1) Declarative programming with recurrent tensors (§3).

To capture dynamic dependencies, we introduce *recurrent tensors* (RTs), a declarative programming model inspired by recurrence equations [13], often used in DL algorithms [9, 11, 14, 31]. RTs can have multiple temporal dimensions (e.g., timestep and iteration), which define a *temporal domain*. RTs can be indexed using symbolic expressions to declaratively express dynamic dependencies. RTs have well-defined domain broadcast [32] and backpropagation [33] semantics.

(2) Optimization using symbolic dependence graphs (§4).

TEMPO translates RTs into a *symbolic dependence graph* (SDG) in which each vertex operator carries the temporal domain of the RT that created it, i.e., the set of timesteps at which the operator produces output tensors; edges describe how operators at different timesteps *depend on* each other's outputs using the symbolic expressions extracted from RTs.

SDGs enable TEMPO to extend classic compiler optimizations (e.g., algebraic simplifications, redundant code elimination) to dynamic computations by manipulating symbolic expressions. In addition, SDGs simplify advanced transformations, such as *vectorization* (batching many small operations) and *tiling* (executing large operations in smaller tiles), by

viewing them as moving tensor dimensions from temporal to spatial and vice versa. For efficient code-generation, TEMPO tiles dynamic dependencies into static-size tiles and *fuses* islands of static operations into single operations.

(3) Scheduling using a polyhedral model (§5.1). The declarative RT program does not prescribe an execution order, and, due to their dynamic dependencies, SDGs cannot be scheduled using topological sorting. Instead, TEMPO uses the polyhedral model [34, 35] for program-wide scheduling: TEMPO translates the SDG into a set of constraints and solves an integer linear program to obtain a schedule that assigns a physical execution time to each timestep. Based on the schedule, TEMPO generates an imperative program for the user computation, represented as an abstract syntax tree (AST).

(4) Automated memory management (§5.2). TEMPO introduces a simple method for automatic memory management, including buffer deallocations, donations, and GPU/CPU buffer swapping. It augments the SDG with explicit memory management operations, which are restricted by dependency edges. Therefore, memory management is also handled when solving for an execution schedule.

Our prototype implementation of TEMPO¹ supports single-GPU execution on multiple execution engines (Torch and JAX). To execute SDGs, TEMPO picks physical tensor memory representations based on dependence expressions and applies existing code-generators [36] to fused operators. It then follows the schedule AST, evaluating symbolic dependence expressions, to resolve dynamic dependencies at runtime.

Our evaluation shows that TEMPO effectively compiles dynamic DL programs, while automating algorithm-specific scheduling and memory management. For LLM decoding using Llama-3.2-3B [37], we compare TEMPO against Torch and JAX and show that it is up to 7× faster and scales to 4× longer sequences due to its awareness of dynamic dependencies. For three typical RL algorithms [14, 15, 38] with different dynamic dependencies, TEMPO achieves up to 54× faster end-to-end training times than five baseline RL systems by eliminating redundant computation. For both workloads, TEMPO has an up to 16× lower peak GPU memory usage due to its tiling optimization and efficient memory management.

¹Available at <https://github.com/LSDS/Tempo>

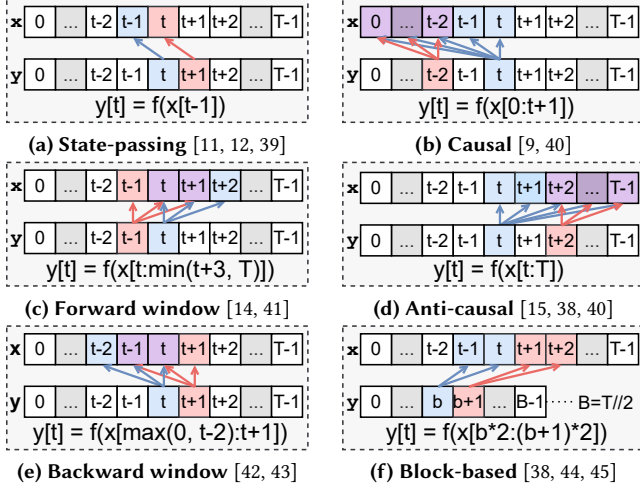


Fig. 2: **Dynamic dependencies** (Tensor y at time t depends on (as indicated by the arrow direction) a dynamic range of x values.)

2 Supporting Dynamic DL Programs

Today’s DL algorithms include diverse dynamic dependencies, often described in papers using recurrence equations [13], such as those in Fig. 2. For example, recurrent [11], state-space [12], and diffusion models [39] all pass state from a timestep to the next (Fig. 2a, where x may equal y). DQN [43] and windowed attention [42] use backwards windows (Fig. 2e). PPO [38], video models [45], and long-context transformers [46] access data in chunks (Fig. 2f). We now focus on two popular workloads and their dynamic dependencies.

Large language models (LLMs) [3, 37] generate text as sequences of tokens. Fig. 3 shows a transformer-based LLM architecture. After an initial *user prompt* processing step (prefill), the LLM begins iteratively *decoding*, i.e., generating a response one token at a time, until it generates a special *end-of-sequence* token at time T . LLMs are stacks of transformer blocks, each with a feed-forward and an *attention* layer.

Attention uses dynamic computation. It first projects the input x_t into query q_t , key k_t , and value v_t vectors, and then computes attention (Eq. 1), where s is a scaling constant and $\phi(t)$ is called the *attention pattern*. This pattern defines a dynamic range of keys and values used to compute attention at timestep t . Typical patterns include causal attention [9] (Fig. 2b) where $\phi(t) = 0:t+1$, and window attention [42] (Fig. 2e) where $\phi(t) = \max(0, t-w):t+1$, with window size w . The dynamic section ends when the dot product with values contracts the dynamically-sized dimension.

$$a_t = \text{softmax}(q_t k_{\phi(t)}^\top \cdot s) v_{\phi(t)} \quad (1)$$

Reinforcement learning (RL) [10] learns an *acting policy* π_W , represented by a deep neural network (DNN) with parameters W , from *experience* data collected by trial-and-error interactions with a simulated *environment* (see Fig. 4). At each *timestep* t , starting from the initial observation o_0 , the *acting policy* produces an action a_t , which is executed in

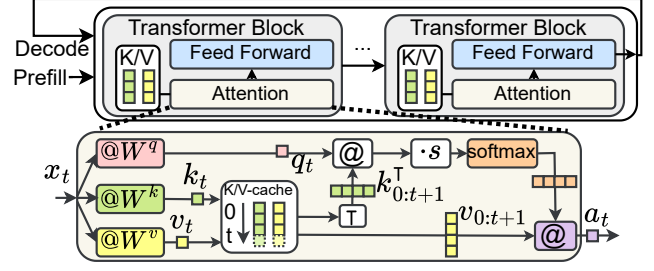


Fig. 3: **Simplified Llama-3.2-3B architecture** (Each decoding step depends on a dynamic range of cached key-value (K/V) pairs.)

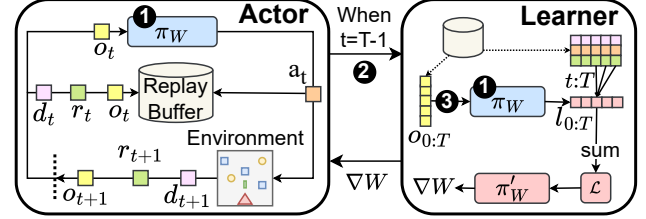


Fig. 4: **Actor-learner architecture in RL systems** (Decoupling acting and learning causes ① duplicate forward passes, ② serial acting and learning, and ③ high peak memory usage.)

the environment and, in response, receives the next observation o_{t+1} , a done signal d_t , and a reward r_t .

In each iteration, a *learner* computes a loss $\mathcal{L}$ from the generated experience data and backpropagates it into a parameter update ∇_W . The loss is a sum of per-timestep losses, $\mathcal{L} = \sum_{t=0}^T l_t$, where each l_t *dynamically* accesses only a *portion* of the experience data, depending on the algorithm. REINFORCE [15] accesses all future steps (Fig. 2d), while A2C [41] uses a forward-looking window (Fig. 2c).

2.1 Dynamic DL and whole-program compilation

Dynamic algorithms are difficult to represent and optimize in today’s DL systems, preventing whole-program compilation. *Imperative (eager)* systems, such as PyTorch [6] and TF-Eager [21], support dynamism well: users write imperative code that dynamically dispatches GPU kernels that are appropriately-sized to the dynamic shape [22]. They rely on the host language (e.g., Python) to evaluate control-flow, conditionals, and dynamic dependencies. These systems, however, lack a graph representation that can be optimized and scheduled ahead-of-time, thus placing the burden of writing efficient code on the user. *Lazy eager* approaches [47] collect small subgraphs before optimizing them, but this is still insufficient for whole-program compilation.

Declarative (graph-based) systems, such as TensorFlow [18] and JAX [7], first build a dataflow graph of the program before performing optimizations and execution scheduling. TensorFlow uses a flat dataflow graph with several control-flow operators [48] that encode loops and conditionals; JAX uses a single while operator [49], with nested subgraphs for loop body and condition. Both approaches, however, require static tensor shapes in the graph, and thus cannot directly express dynamic data dependencies.

As a result, users are forced into different workarounds to implement dynamic computation, each with drawbacks:

(a) Per-shape compilation. Users can compile a separate graph for each possible tensor shape, but this is expensive for highly variable shapes [47] as found in LLMs and RL.

(b) Unrolling loops. Users can statically unroll the timestep loop [7] when T is known, generating all instantiations of dynamic dependencies. This leads to *graph size explosion*, increasing compilation times, and also loses the semantics of dynamic dependencies, which prevents optimizations.

(c) Padding and masking. Users can coarsen dynamic dependencies, padding them to a fixed maximum sequence length $T' > T$. For correctness, users must also mask out valid timesteps. This is cumbersome and introduces processing and memory overheads.

2.2 Consequences in DL implementations

We now discuss how the lack of support for dynamic dependencies affects the implementations of DL algorithms.

In **LLM inference**, the dependence on past key/value (K/V) pairs hinders whole-program compilation. This dependence creates dynamic tensor shapes, which require inefficient padding to compile. Furthermore, to avoid recomputing keys and values, K/V caches are used (see Fig. 3), often managed by external systems [50, 51].

Managing these caches is complex because different types of attention require different cache policies. While causal attention [9] must store all prior K/V pairs, window attention [42] allows for the deallocation of old K/V pairs. Linear [52], global [42], block [46], and full attention [53], or hybrid models that interleave attention types [54], demand even more nuanced cache policies. Since existing LLM inference systems [50, 55] do not model dynamic dependencies, they struggle to offer tailored cache policies,² requiring large engineering effort for each new supported attention type.³

RL frameworks [56–62] address forward-looking dependencies in loss computation by coarsening them to $0:T$ and decomposing the computation into two subgraphs: (i) an *actor* graph executed repeatedly until termination, which produces the full simulation data and stores results in a *replay buffer* [63, 64]; and (ii) a *learner* graph that is invoked once after the simulation terminates to compute a parameter update using the experience from the replay buffer.

While the actor-learner model can be applied to most RL algorithms, it has several drawbacks (indicated by the numbers in Fig. 4) due to the independently compiled subgraphs:

(1) Redundant computation. Intermediate activations produced by the actor π_W are discarded and recomputed in the learner for use in back-propagation. This adds substantial overhead, as it doubles the amount of inference work per timestep for typical algorithms;

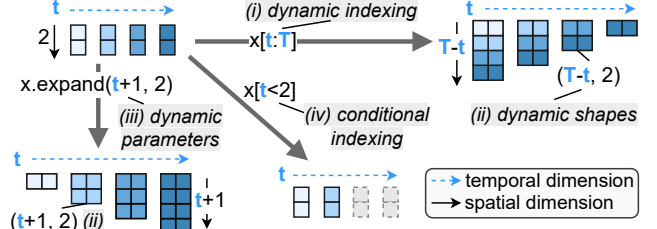


Fig. 5: How a RT x with domain $(t,)$ can be used in TEMPO

(2) Missed parallelism. The execution of actors and learners must be serialized due to the coarse $0:T$ dependency. This prevents the parallelization of learning on past timesteps while acting out future timesteps; and

(3) High peak memory usage. The “all-at-once” learning prevents early deallocations and results in high peak GPU memory usage, potentially causing out-of-memory errors.⁴

3 Recurrent Tensors

While writing dynamic algorithms is challenging in current DL systems (see §2.1), we observe that they are naturally expressible as *recurrence equations* [13]. This makes recurrence equations ubiquitous in algorithmic pseudocode, e.g., when defining optimizers [65], learning rate schedules [66], models [39, 67, 68], layers [9, 42], and loss functions [31, 38].

To exploit this expressiveness, TEMPO introduces *recurrent tensors* (RTs). In addition to the spatial dimensions for its shape, an RT has *temporal dimensions*, such as iterations or timesteps, which define its *domain* (see Fig. 5). Each temporal dimension has an associated pair of symbols—a *current step* (e.g., t, i) and an *upper bound* (e.g., T, I). If an RT x has t in its domain, it is defined by a recurrence equation for every point $0 \leq t < T$, and its value (and shape) may vary with t . RTs enable declarative programming, as their defining equations can refer to past or future values of other RTs by indexing them with *symbolic expressions*.

Symbolic expressions are constructed from symbols using overloaded operators, such as arithmetic (e.g., $+$, $/$), boolean comparison (e.g., $\leq$, $=$) and logic (e.g., $\&$, $|$), as well as $\min$ and $\max$. Accessing a range of temporal values is done using a *slice expression* (start:stop:step), and to express multiple dimensions at once, *sequence expressions* ($[\cdot, \cdot]$) are used.

Symbolic expressions are versatile and can be used to (as shown by the labels in Fig. 5): (i) index the temporal dimensions of RTs expressing dynamic dependencies; (ii) express dynamic shapes; (iii) parameterize tensor operations which receive index or shape arguments (e.g., a dynamic expand or spatial index select); and (iv) naturally express conditional computation using boolean expressions. Indexing an RT’s temporal dimension with a slice expression creates a (possibly dynamic) leading spatial dimension in the output RT.

²<https://github.com/vllm-project/vllm/issues/15705>

³<https://github.com/vllm-project/vllm/pull/14097>

⁴Consider learning from small $W=H=256$ images with $C=3$ color channels, at $P=4b$ float precision, with a batch size of $B=512$, and $T=1000$ timesteps. This requires at least $T \times B \times (W \times H \times C \times P) / 2^{30} = 375$ GB of memory.

Alg. 1: REINFORCE using RTs. (Line 13 is discussed later.)

```

1  import tempo as tpo
2  ctx = tpo.Context(num_dims=3)
3  with ctx as (b, B), (i, I), (t, T):
4      env = tpo.rl.env.make("gym.CartPole-v1")
5      dnn = tpo.DNNBuilder(domain=(i,))
6          .from_env(env, hidden=[32, 32]).build()
7
8      o = tpo.like(env.obs_space, domain=(b, i, t))
9      o[b, i, 0] = env.reset(domain=(b, i))
10     a = dnn(obs) # Acting
11     o[b, i, t+1], r, d = env.step(a)
12
13     # Different access pattern -> different schedule
14     - g = r[b, i, t:T].discounted_sum(0.95)
15     + g = r[b, i, t:min(t+5, T)].discounted_sum(0.95)
16     l = -dnn.log_prob(a) * g # Learning
17     l[0:B, i, 0:T].mean().backward()
18     lr = 1e-3 * (0.99 ** i) # Decaying learning rate
19     tpo.optim.Adam(dnn.params, lr=lr).step()
20
21     dnn[(i+1) % 5 == 0].checkpoint(save_path)
22     ctx.run({B: 64, I: 20, T: 200})

```

REINFORCE with RTs. Alg. 1 shows an example of the REINFORCE RL algorithm [15] written in TEMPO. Users begin by setting up a context (line 2) with a chosen number of temporal dimensions⁵, associating each dimension with a semantic meaning (line 3): batch sample b , iteration i , and timestep t . An RL environment is created (line 4), and a DNN is defined (line 5). Similar to PyTorch [6], DNNs are defined by composing modules, but the domain over which parameters vary (iterations) must be explicit (line 5).

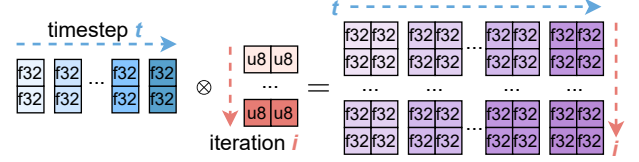
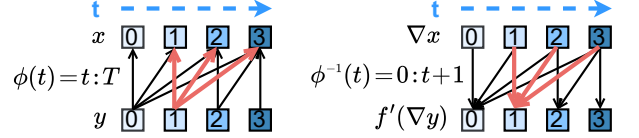
The observations tensor is defined as a *branching* RT: the first timestep is given by the environment reset (line 8) and subsequent timesteps are computed by stepping the environment with the DNN’s action (line 10). The *returns* g of each timestep t are computed from the rewards r indexed by $[t:T]$ (line 12), yielding an RT with *dynamic shape* $(T-t,)$. TEMPO propagates dynamic shapes transparently.

The loss is then backpropagated (lines 14–15), and the gradients are used by the Adam optimizer [65] (line 17), also defined using RTs, to update the DNN. Expressing conditional computation, such as checkpointing (line 18), is done by indexing with boolean expressions, which only execute when the boolean evaluates to true. Finally, the program is compiled and run, with the user providing bounds for each temporal dimension (line 19).

Note how, with RTs, there is no need to decouple acting and learning. The original forward pass can simply be used within the loss definition. Furthermore, there is no need to use replay buffers [63] to store prior timesteps of experience⁶, because TEMPO manages the storage of each RT based on the dynamic dependencies.

⁵With zero temporal dimensions, TEMPO behaves like a typical DL system with only spatial tensor dimensions.

⁶A replay buffer is still needed for asynchronous [41] or offline RL [69].

**Fig. 6: Domain unioning** (Similar to shape broadcasting and datatype promotion, domains are unioned when tensors interact.)**Fig. 7: Symbolic automatic differentiation** ($y[1]$ depends on $x[1:4]$ and thus $\nabla y[1]$ contributes to $\nabla x[1:4]$.)

Automatic domain inference. Annotating domains is often unnecessary, as TEMPO infers them. For example, temporally indexing an RT with a constant (e.g., 5 or $0:T$) on dimension t removes t from the resulting RT’s domain. Spatially indexing an RT with t adds t to the resulting RT’s domain. If an RT is not explicitly temporally indexed (g in line 14), it is treated as if indexed by the identity (i.e., $g[b, i, t]$).

When two RTs interact, their domains are *unioned*, as shown in Fig. 6. In this example, the left RT varies with timesteps t , while the right RT varies with iterations i . Their interaction produces an RT that varies with both (i.e., has a domain of (i, t)). Domain inference allows, e.g., for learning rate schedules, to be easily expressed as RTs (line 16), and be integrated with the optimizer without code changes.

Symbolic automatic differentiation. When `backward()` is called on an RT, automatic differentiation [33] is used to compute the input gradients for an operator based on its output gradients. TEMPO, however, must also accumulate gradients through temporal dimensions. Consider the RT $y[t] = f(x[\phi(t)])$, where ϕ is called the *dependence expression* y has on x . A single timestep of x may contribute to several timesteps of y . Thus, the gradient of x , ∇x , is an RT with the same shape, datatype, and temporal domain as x , which sums the gradient contributions from all timesteps of ∇y to which $x[t]$ contributed to:

$$\nabla x[t] = \sum_{t' \in \phi^{-1}(t)} f'(\nabla y[t'])$$

where f' is the derivative of f , and ϕ^{-1} is the *inverse* of the dependence expression. Inverting a dependence can be thought of as converting “which source timesteps does the sink depend on at timestep t ?” to “which sink timesteps depend on the source’s timestep t ?”. For example, $[t+3]$ inverts into $[t-3]$, $[\max(0, t-3) : t+1]$ into $[t : \min(t+4, T)]$, and $[t:T]$ into $[0:t+1]$. This last example is depicted in Fig. 7. If y indexes x with a constant expression (e.g., $y = f(x[0:T])$, y does not vary with t . To recover ∇x ’s domain, TEMPO must select the corresponding spatial dimension of y using t as an index, i.e., $\nabla x[t] = f'(\nabla y[\]).\text{index}(t)$. This process generalizes to multiple temporal dimensions by concatenation.

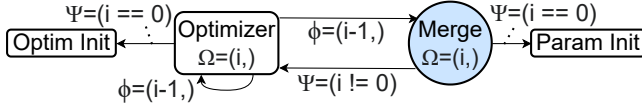


Fig. 8: Encoding DNN state using cycles (Boxes are subgraphs.)

4 Symbolic Dependence Graphs

As users define DL programs with RTs, TEMPO constructs a *symbolic dependence graph* (SDG) of the program, which encodes the dynamic dependencies between operators.

4.1 SDG representation

An SDG is a directed graph of *operators*. Each operator may have multiple input or output tensors, and cycles are allowed. Each operator o is tagged with a temporal domain $\Omega(o)$: the set of temporal *points* in $\mathbb{Z}^n$ at which the operator executes, producing output tensors.

Edges describe how operators at different temporal points depend on each other's outputs. Each *edge* $o_1 \xrightarrow{\phi} o_2$ is annotated with a *dependence expression* ϕ . It indicates that o_1 (the sink) at point $p \in \Omega(o_1)$ *depends on* o_2 (the source) at points $\phi(p)$. The domain Ω and dependence expressions ϕ are obtained directly from the RTs that defined them.⁷

Tensor operators. TEMPO supports a minimal set of 44 stateless operators, most of which are typical elementwise maps, reductions, scans, view, or spatial indexing operators. View and indexing operators (e.g., ReshapeOp, SliceOp) may use symbolic expressions in shape and index parameters. UDFOps allow users to register custom operations, useful for injecting (retrieving) values to (from) the runtime. UDFOps may access *external state*. TEMPO uses UDFOps to integrate with RL environments [70] and LLM tokenizers [71].

To support branching RTs, we define MergeOps. They conditionally select which of their multiple inputs to *move* to their single output based on which edge condition ψ first evaluates to true at time t . Edge conditions are obtained from the conditional indexing of RTs.

Encoding state. To encode state (e.g., DNN parameters) in a graph, Tensorflow [18] uses stateful operators (`tf.Variable`). JAX [7] graphs are stateless, and JAX offloads state management responsibilities to the user. In TEMPO, state is instead encoded in the SDG using MergeOp cycles.

Fig. 8 shows an example of this for a DNN parameter. The parameter's initial value is given by a parameter initialization subgraph. Subsequent parameter values are obtained from the optimizer step subgraph, which itself depends on the previous parameter value to compute an update. Optimizers such as Adam [65] also hold state, which is encoded using the same mechanism. This approach enables TEMPO to represent state implicitly, without additional operator types that have complex semantics and side-effects, and without requiring users to manage state explicitly.

⁷Throughout the paper, to simplify notation, we use t for generic temporal dimensions and omit *identity* dependencies (i.e., $\phi(t) = t$) in figures.

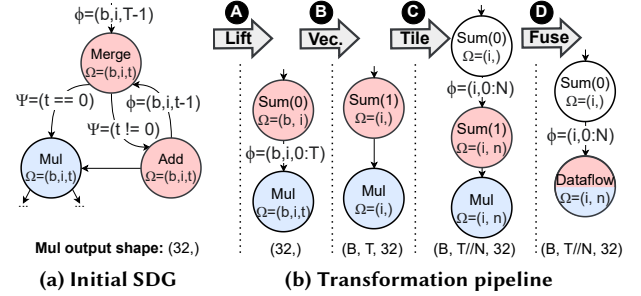
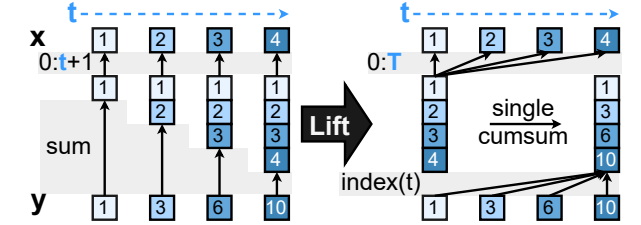


Fig. 9: SDG passing through TEMPO's transformation pipeline

Fig. 10: Lifting shown with RTs (left: redundant sum of $0:t+1$ at each t ; right: equivalent with single cumulative sum of $0:T$)

Optimizing symbolically. TEMPO applies compiler optimizations to SDGs, such as dead and duplicate code elimination, algebraic equivalences, and broadcasting removal, which are extended to support dynamic dependencies.

Suppose that TEMPO considers a re-association optimization for a chain of matrix multiplications $(A \cdot B) \cdot C$, where $A \in \mathbb{R}^{32,16}$, $B \in \mathbb{R}^{16,t}$, and $C \in \mathbb{R}^{t,8}$. Since t is in the shape of B and C , $(A \cdot B) \cdot C$ has a dynamic cost of $32 \cdot 16 \cdot t + 32 \cdot t \cdot 8 = 768 \cdot t$ FLOPs. Re-associating as $A \cdot (B \cdot C)$ costs $16 \cdot t \cdot 8 + 32 \cdot 16 \cdot 8 = 128 \cdot t + 4096$, which is lower when $t > 6.4$. TEMPO can use the original association for $t \leq 6$, and re-associate when $t > 6$.

TEMPO also performs *domain reduction*, which reduces the domain of operators to the union of the domains of their dependencies, thus removing redundant work. This is akin to *loop-invariant code motion* [72] in optimizing compilers.

The SDG in Fig. 9a computes the sum of the MulOp's output over timesteps t using a MergeOp. At $t=0$, the MergeOp simply copies the MulOp's output; otherwise, it adds the current MulOp output to the sum at $t-1$. Though correct, this SDG is inefficient. Fig. 9b shows how TEMPO progressively *transforms* this SDG for efficient execution.

Lifting. TEMPO starts by eliminating *recurrent patterns* (reductions, scans [73], and stencils [26] implemented using MergeOps) that prevent vectorization. SDGs enable simple pattern-matching of these complex structures, allowing TEMPO to replace them with batch SumOps, CumSumOps, or ConvOps. In Fig. 9b, TEMPO *lifts* **A** the recurrent addition into a SumOp.

Lifting can also optimize computation. In Fig. 10, the original computation, $y[t] = x[0:t+1].\text{sum}()$, redundantly sums the prior steps of x at each timestep t . TEMPO replaces this with $y[t] = x[0:T].\text{cumsum}()$, and restores the temporal dimension t using a spatial indexing operation with t as an index.

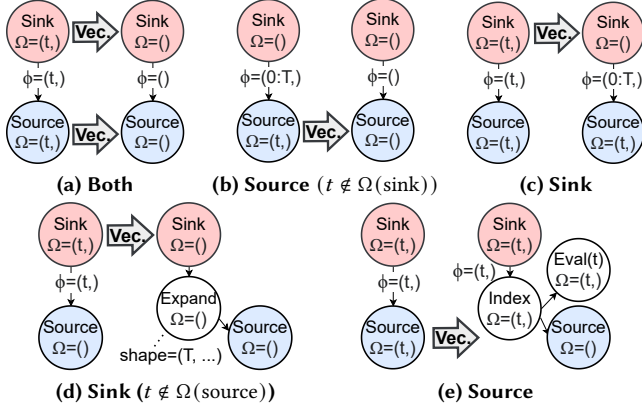


Fig. 11: How TEMPO updates edges when vectorizing

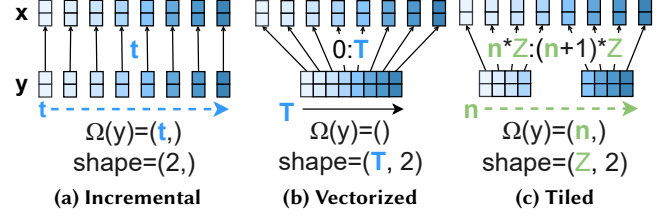
4.2 Vectorization

Vectorization (B in Fig. 9b) reduces the execution time of SDGs by laying out temporal dimensions spatially. Vectorized operators execute fewer times but with larger input and output tensors. This offers greater SIMD parallelism [74], at the cost of increased peak memory usage.

For each temporal dimension t , TEMPO finds a set of vectorizable operators and applies a *vectorization rule* to each. Such rules are simple: most operators simply have t removed from their domain and T prepended to their input and output shapes; operators with dimension parameters (e.g., SumOp, SqueezeOp) have them incremented, to account for T being prepended to their input shape; and operators with shape parameters (e.g., ReshapeOp, ExpandOp) also prepend T to this shape. When vectorizing IndexOps or IndexAddOps and both their source (i.e., the tensor being indexed) and index input tensors are also being vectorized, TEMPO must promote these operators into GatherOp and ScatterAddOp, because index operations only support one-dimensional indexes.

Vectorizability. Cycles complicate vectorization, because incorrectly vectorizing them can lead to *unschedulable* SDGs. TEMPO avoids unschedulable SDGs using the following conservative approach: we define a cycle as *trivial* on t if all symbolic indexes of that dimension are $[t]$ itself or a constant expression (e.g., $[0 : T]$, $[5]$, $[0 : 5]$). TEMPO does not vectorize operators in non-trivial cycles; operators in trivial cycles must all be vectorized or none can be.

Handling edges. TEMPO must also update SDG edges for correctness (see Fig. 11). If both source and sink are vectorized (Fig. 11a) or, if only the source is, and the sink does not have t in its domain (Fig. 11b), TEMPO drops the t index from the connecting dependence ϕ . If only the sink is vectorized, the t index is promoted to $0 : T$ (Fig. 11c). If the sink is vectorized, and the source does not originally vary with t (Fig. 11d), the source shape is expanded to the T elements expected by the sink. Finally, if both vary with t , but only the source is vectorized (Fig. 11e), an IndexSelectOp is added to extract the t -th spatial element each step t .

Fig. 12: Equivalent execution strategies for $y[t] = -x[t]$ as RTs (Example assumes x is not also vectorized or tiled.)

4.3 Tiling

Some operators require too much memory, especially after vectorization. In these cases, TEMPO can *tile* the SDG (C in Fig. 9b). Similar to vectorization, tiling computes a set of operators to tile, applies a tiling rule to each, and updates the edges. Unlike vectorization, however, tiling adds a new temporal dimension n to the domain of tiled operators and decomposes a chosen spatial dimension of size D into N tiles of size Z , with $N = D/Z$.

Computing the tiled operator set. Since reductions (e.g., SumOp and MatMulOp's contracting dimension) eliminate specific input spatial dimensions, they are natural starting points for tiling. TEMPO picks one of the reduced spatial dimensions, D , to tile on, preferring dimensions which are eventually introduced by temporal indexing operations. This is because tiling on these enables online garbage collection during scheduling (§5.2) and thus implicitly enables advanced execution strategies, such as *gradient accumulation* [75].

Next, TEMPO recursively traverses operator dependencies, adding them to the tiled operator set until it encounters one of the following stopping conditions:

- a dependence expression ϕ that creates the spatial dimension being tiled. In this case, TEMPO modifies the expression to instead temporally access the n -th tile of size Z ;
- an operator with a memory demand below a user-defined threshold. Here, TEMPO inserts a SliceOp to spatially slice the n -th tile of size Z from the non-tiled dependency;
- an ExpandOp that expands the dimension being tiled. TEMPO modifies the ExpandOp to expand to Z instead of D .

A tiling rule is then applied to each operator in the tiled operator set. These are essentially the opposite of the corresponding vectorization rules. To finish the tiling, TEMPO performs an aggregation over all K tiles (e.g., the white sum of C in Fig. 9b), restoring the original reduction.

Fig. 12 visualizes vectorization and tiling as RTs. Initially defined by a user *incrementally* (Fig. 12a), the identity dependency allows $y[t]$ to execute immediately after $x[t]$. TEMPO can also vectorize y (Fig. 12b), which requires that y wait for all $x[0 : T]$ values, increasing memory usage. TEMPO can then tile the freshly vectorized dimension (Fig. 12c). Note how $y[n]$ now accesses a tile of size $Z = 4$ using the dependence expression $\phi = [n \cdot Z : (n+1) \cdot Z]$. Fully tiling on the same spatial dimension brings the SDG back to Fig. 12a.

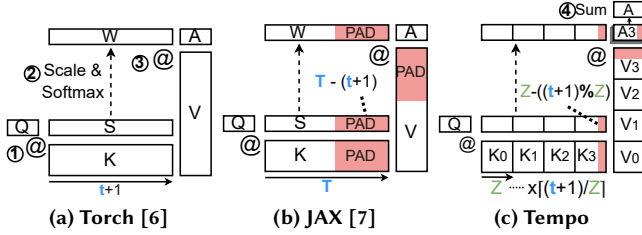


Fig. 13: Attention in DL systems (Torch computes attention eagerly. JAX optimizes it, but introduces large padding. Tempo tiles it into static tiles, enabling code-generation with minimal padding.)

Static tiling. In general, dynamic dependencies with changing shapes prevent code-generation. In these cases, TEMPO tiles them into *statically-sized tiles* by adding minimal padding.

Fig. 13 shows this for causal attention (Eq. 1), which uses $K[t] = k[0:t+1]$ to obtain dynamically shaped K and V tensors. As discussed in §2.1, Torch [6] eagerly executes attention despite the dynamic shapes (Fig. 13a), which prevents optimizations; JAX [7] requires padding to a static upper bound T to enable whole-program optimization, which adds overhead that grows with T (Fig. 13b).

In contrast, TEMPO tiles attention, trading a dynamic computation for a dynamic number of static tiles (Fig. 13c). Tiling starts from $W \cdot V$, which contracts the dynamic dimension, and continues through the softmax, down to K and V . This creates $\lceil (t+1)/Z \rceil$ tiles of size Z for each tensor.

TEMPO uses $K[t, n] = k[n \cdot Z : \min((n+1) \cdot Z, t+1)]$ to construct each tile, which may not fill Z timesteps due to the use of $\min$. Thus, it is necessary to pad the remaining amount: $Z - (t+1) \bmod Z$. TEMPO also appropriately masks this padding, using a 0 mask for MatMulOps and SumOps, and a $-\infty$ mask for MaxOpOp. To obtain the final attention output, TEMPO then sums together each masked attention tile.

Padding and masking overhead is minimal, as they are only applied to the last tile. Furthermore, TEMPO can often optimize them away by hinting to the runtime that buffers for K and V tiles should be pre-allocated with the padded size and pre-filled with the mask value (see §6).

4.4 Operator fusion

An SDG often contains static *islands* in which all operators have the same domain and only identity edges between them. Within these islands, the SDG directly passes the outputs of one operator to the inputs of another. The *fusion transformation* identifies and *fuses* (⑤ in Fig. 9b) these islands into a single DataflowOp operator, routing edges as needed.

TEMPO initially assigns each operator to its own island and iteratively merges islands with the same domain and no inter-dependencies. Dynamic operations (e.g., RNGOp, UDFOp, MergeOp) are excluded from fusion, because they cannot be compiled statically. Fusion enables TEMPO to leverage existing DL code-generators [7, 16, 76] to generate kernels. This lowers dispatching overheads, minimizes the number of tensors TEMPO must manage, and speeds-up scheduling.

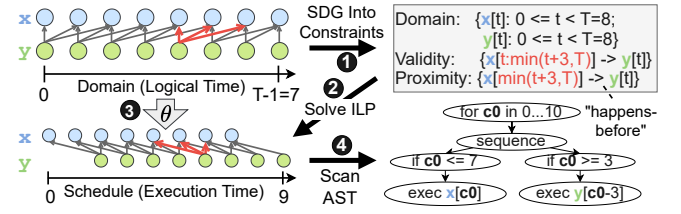


Fig. 14: How TEMPO schedules $y[t] = f(x[t: \min(t+3, T)])$

5 Scheduling SDGs with the Polyhedral Model

RTs are declarative and thus require scheduling to find an execution order. Due to their dynamic dependencies, existing dataflow scheduling techniques [18, 19], however, are unsuitable. TEMPO uses a *polyhedral model* [34, 35] to address this challenge. Scheduling occurs in two rounds: TEMPO first finds a schedule for tensor operations (§5.1), and then uses it to schedule memory management operations (§5.2).

5.1 Polyhedral execution scheduling

The polyhedral model [34, 35] has its origins in the scheduling of uniform recurrence equations [13], but has since evolved to support more complex computation [77]. Today, it is used to optimize localized loop nests (kernels) in imperative programs [28, 78], as using it to model arbitrary programs remains challenging [79]. Since RTs are inspired by recurrence equations, TEMPO can apply the polyhedral model for whole-program scheduling.

Scheduling in a polyhedral model finds a *schedule function* θ that maps each n -dimensional timestep of an operation to an *execution time* in $\mathbb{Z}^s$, $s \geq n$, and where the lexicographic order⁸ ($\prec$) determines what operation executes first. A *valid* schedule must respect all dependencies, i.e., for $y[t] = f(x[\phi(t)])$ then $\forall t \in \Omega(y). \theta(x[\phi(t)]) \prec \theta(y[t])$. Finding a schedule function requires solving an integer linear program (ILP) [34], with an ILP objective which minimizes the sum of *dependence distances* $\theta(y[t]) - \theta(x[\phi(t)])$ constrained by a set of *validity constraints* derived from program dependencies. This yields schedules with low makespan.

Fig. 14 shows the scheduling approach in TEMPO. SDGs already represent tensor domains, dependencies and conditions explicitly using symbolic expressions, which can be translated easily into polyhedral validity constraints (① in Fig. 14). In addition, TEMPO creates *proximity constraints*, which guide the scheduler towards schedules with higher temporal locality and smaller tensor lifetimes. TEMPO uses the *Pluto* scheduler [35] through the *isl* library [34] (②). In Fig. 14, the schedule found delays y 's execution by 3 steps.

Since TEMPO fuses large regions of the SDG, the number of operations is reduced, making the ILP tractable even for large programs. TEMPO stores the resulting schedule function θ (③) for scheduling memory management operations, as discussed next. We describe how TEMPO scans the schedule to produce an imperative program AST (④) in §5.3.

⁸Lexicographic order compares tuples left to right: $(1, 2) \prec (1, 5) \prec (2, 1)$.

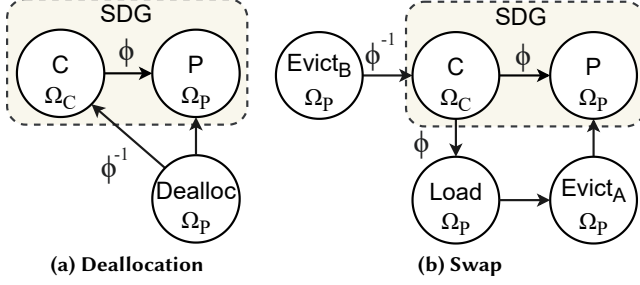


Fig. 15: SDG memory management augmentations

5.2 Memory management

Memory management is a key concern in DL systems [20, 80, 81]. Using the polyhedral model, TEMPO automates memory management concerns: (i) deallocations, (ii) GPU/CPU memory swapping, and (iii) buffer donations.

The key idea is to *augment* the SDG with memory management operations (and constraining dependencies), which enable TEMPO to reuse the polyhedral scheduler to find a memory management schedule.

Scheduling deallocations. Fig. 15a shows how TEMPO augments the SDG with deallocations for the tensor produced by P and consumed by C . Since P produces a tensor for each timestep in its domain, Ω_P , the deallocation Dealloc shares this same domain. TEMPO interprets the execution of $\text{Dealloc}[t]$ as the time to deallocate the tensor $P[t]$.

Deallocation must always happen after production, which TEMPO models with an identity dependence, $\text{Dealloc} \rightarrow P$. Then, to also ensure $P[t]$ is deallocated only after the consumer has finished using it, TEMPO adds a dependence from Dealloc to C , with the inverse dependence expression C has on P . For example, if $\phi = t - 3$, then $\text{Dealloc}[t]$ must wait for $C[t + 3]$ to run, before it can deallocate $P[t]$.

Scheduling GPU/CPU memory swapping. Fig. 15b shows how TEMPO augments the SDG with evict (GPU-to-CPU) and load (CPU-to-GPU) memory movement operations. TEMPO inserts an initial evict operation Evict_A , which evicts the tensor $P[t]$ immediately after production. It then inserts Load and Evict_B operators, to re-load and again evict all timesteps of P needed by C at timestep t . All operations have domain Ω_P and TEMPO interprets the execution of $\text{Evict}_{A/B}[t]$ and $\text{Load}[t]$ as the time to evict or load $P[t]$, respectively.

A tensor can only be evicted after it has been produced, and loaded after it has been evicted. TEMPO models this with the identity dependencies, $\text{Load} \rightarrow \text{Evict}_A \rightarrow P$. To guarantee that the timesteps of P needed by $C[t]$ are available in-memory when $C[t]$ executes, TEMPO adds $C \xrightarrow{\phi} \text{Load}$. Finally, similarly to deallocations, Evict_B must execute only after the last use of $P[t]$ by C , so TEMPO adds the inverse of the consumer's dependence from Evict_B to C .

By placing these dependencies, TEMPO introduces constraints on when the memory management operations can execute, which guarantees that the resulting schedule is

valid and efficient. To prevent thrashing [82], TEMPO does not currently swap frequently accessed tensors, such as DNN parameters or K/V caches in attention models. At runtime, TEMPO performs such swap operations *asynchronously*, through a shared *page-locked* host-memory manager, which pre-allocates page-locked buffers for each swapped RT.

Finding buffer donations. Given the SDG's stateless nature, it is also important to detect when a tensor produced by one operation can be *donated* to be reused by another operation. This avoids expensive re-allocations and enables in-place mutation of, e.g., DNN parameters.

After a tensor is donated, it is invalidated and can no longer be used by other operations. Thus, an operator O_d can only donate its output tensor to one of its dependents, called the *donation receiver* O_r , if for every timestep t in its domain $\Omega(O_d)$, O_r is the last user of $O_d[t]$. This requires that it is scheduled *after* all other competing dependents O_c according to the schedule function θ . Formally, TEMPO donates O_d to O_r if and only if

$$\forall t \in \Omega(O_d). \forall O_c \in \mathcal{D}(O_d) \setminus \{O_r\}. \theta(\phi_c^{-1}(t)) < \theta(\phi_r^{-1}(t))$$

where $\mathcal{D}(O_d)$ is the set of dependents of O_d , ϕ_c^{-1} and ϕ_r^{-1} are the inverse dependence expressions from O_c and O_r to O_d respectively, and $\setminus$ is the set difference operator.

5.3 Generating imperative programs

TEMPO must obtain an imperative program for the user's declarative RT computation, but the schedule function θ produced by the polyhedral scheduler [35] is not directly executable. TEMPO transforms it into an abstract syntax tree (AST) by using the polyhedral library [34] to scan [83] it. The AST contains the following types of statements:

- **sequence:** a block of AST nodes executed in order;
- **if:** a conditional with then and else sub-ASTs;
- **loop:** a generic loop with an iteration counter, start value, condition, step and body sub-AST;
- **execute:** executes operation $O[t]$, where t is given by a mapping from loop counters; and
- **deallocate/load/evict:** executes the memory instruction for timestep t (or multiple steps at once) given by a mapping from loop counters.

To generate efficient ASTs, TEMPO applies further optimizations: (1) it *partially unrolls* loops to reduce the number of nested conditionals; (2) it *promotes* static loops that contain only memory management operations into batched instructions by mapping the loop's range to a slice of points; and (3) it finds and removes redundant loads/evicts by removing (i) any evict followed by a load or deallocation for the same tensor; and (ii) loads for already in-memory tensors.

By interpreting the AST at runtime, users are able to step through the program with high granularity. This allows for simpler debugging and integration with existing DL libraries.

6 Execution Runtime Implementation

The core of TEMPO is written in ~33K Python LoC. Its runtime (rightmost block in Fig. 1) is responsible for executing the program given an SDG, imperative AST, and DL backend.

The runtime executor is simple: it traverses the schedule AST from the root, keeping track of loop counters as they are introduced, evaluates if conditions and follows each execute/deallocate/swap instruction in order. We highlight four key components of the runtime:

DL execution backends implement a thin interface, which has methods for (i) allocating tensors, (ii) moving tensors between GPU/CPU, (iii) translating each TEMPO tensor operation to the backend equivalent, and (iv) a code-generation function that returns a kernel when given a fused DataflowOp. TEMPO currently offers Torch [6] (without buffer donation support) and JAX [7] DL backends, easing integration with existing projects.

Kernel launchers manage the inputs and outputs of kernel invocations. When the executor encounters an execute instruction, it invokes the corresponding kernel launcher with the loop counter state. Using the loop counters, kernel launchers evaluate input dependence expressions and use the result to index tensor stores for the inputs to the kernel. They then execute the kernel and store the outputs back in tensor stores at the current timestep t .

Tensor stores are how TEMPO stores the timesteps of RTs at runtime. Tensor stores are always written into timestep-by-timestep, but are accessed using various patterns, which requires tailored storage strategies for best performance. TEMPO assigns each RT to one of three storage methods:

- If the RT is point accessed (see Fig. 2a), TEMPO stores it in a *point store*, which maps the point directly to a physical tensor. This makes point insertions and reads fast, at the cost of requiring tensor stacking for slice reads;
- If the RT is block accessed (see Fig. 2f) TEMPO stores it in a *block store*, which maps the access to a block-sized pre-allocated buffer. It offers fast slice reads, but writes require in-place mutation. Causally and anti-causally accessed RTs also use this store (with buffer size T);
- If the RT is window accessed (see Fig. 2c), TEMPO stores it in a *window store*, which maps the access to a pre-allocated circular buffer twice the size of the window. TEMPO writes to two mirrored locations, ensuring that a contiguous read window is always ready.

Kernel wrappers inject storage related operations into kernels prior to code-generation (see Fig. 16). This enables efficient interaction with block or window stored RTs. Normally, a kernel first produces an output tensor, which is then copied into a pre-allocated buffer at index t . The *in-place writes* wrapper eliminates such copies by enabling the kernel to write directly to the preallocated buffer. Conversely, some DL backends (e.g., JAX) perform copies when slicing tensors.

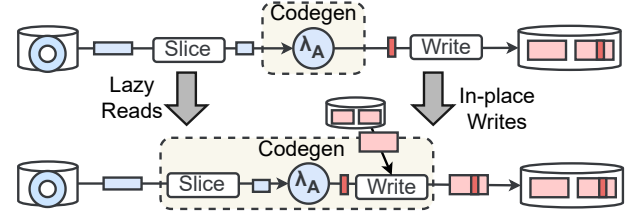


Fig. 16: Kernel wrappers (TEMPO avoids copies by co-designing the storage and code-generation of kernels.)

The *lazy reads* wrapper optimizes this by slicing the backing buffer in the code-generated region.

7 Evaluation

We evaluate TEMPO to answer the following questions: do TEMPO’s optimizations lead to more efficient computation compared to existing approaches? does TEMPO’s tiling and memory management enable scaling on a single GPU? can TEMPO effectively support algorithm-specific execution and memory-management schedules? does TEMPO’s compilation scale to large model sizes?

7.1 Methodology

We evaluate TEMPO on LLM decoding and RL training.

LLM decoding. We focus on decoding, because it often dominates LLM inference latency [84]. We implement a Llama-3.2-3B [37] decoding loop (Fig. 3) in TEMPO, and compare it against end-to-end JIT-compiled JAX [7] (a graph-based system), and Torch [6]⁹ (an eager-execution system). For the JAX baseline, we pad the minimal amount possible (i.e., up to the sequence length being decoded). We compare the performance as we modify batch size, sequence length, and the attention pattern from causal [9] to windowed [42]. All systems use the same tokenizer, sampling method, and prompts.

RL training. We experiment with (a) PPO [38], (b) REINFORCE [15] and (c) REINFORCE with n -step returns [14] (essentially A2C [41]). We compare TEMPO against 5 RL frameworks with different focus: (i) performance-focused frameworks (SampleFactory v2.1.1 [57], RLGames v1.6.1 [61]), (ii) single-file algorithm implementations (CleanRL commit e648ee2 [85]), and (iii) scalable RL frameworks (Ray RLlib v2.5.0 [59]). To focus on framework overheads, we employ a test RL environment [86, 87] with short simulation-step time for all experiments. It is GPU-accelerated except for RLlib, which lacks such support.

All experiments use the same hardware and software configuration. We use a server with an AMD EPYC 7402P 24-core CPU, 384 GB of DDR4 RAM (3200 MT/s) and an NVIDIA RTX A6000 GPU (48 GB of GDDR6 RAM, PCIe Gen4 $\times 16$ at 32 GB/s). We run Ubuntu v22.04 with Linux kernel v5.15, CUDA v12.8, PyTorch v2.7.1, and JAX v0.6.2. We discard the first iteration of each experiment to avoid warmup effects.

⁹We use Meta’s open-source inference implementation from <https://github.com/meta-llama/llama3>.

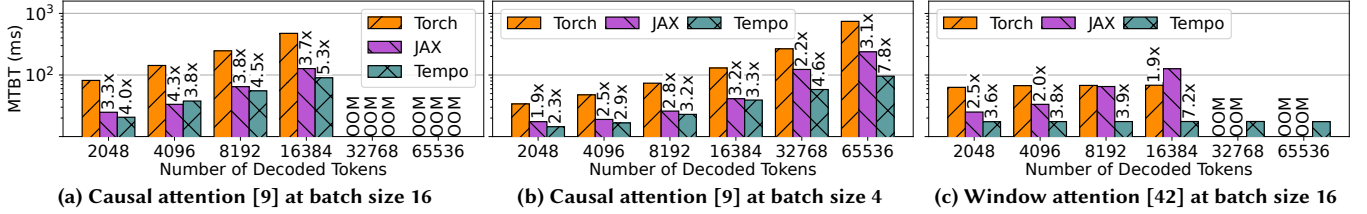


Fig. 17: Llama-3.2-3B mean time between tokens (MTBT) with different attention mechanisms and batch sizes

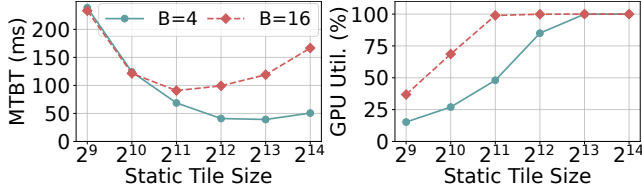


Fig. 18: Effect of TEMPO's tile size on Llama-3.2-3B

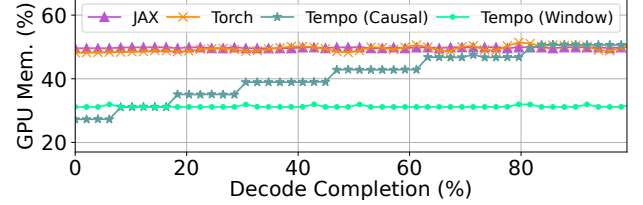


Fig. 19: Runtime GPU memory use for Llama-3.2-3B decoding

7.2 LLM decoding

We begin our evaluation with Llama-3.2-3B [37] token-by-token decoding. For each system, we fix the batch size to 16 and measure the latency in terms of mean time between tokens (MTBT) when decoding sequences of length $T=2,048$ to $T=65,536$ tokens with standard causal attention. Fig. 17a shows the results, where the factors indicate speedups over the slowest system. In general, as we increase the sequence length, the MTBT of all systems increases. This is expected as the dynamic $[0:t+1]$ dependency of causal attention increases the amount of work with each added timestep.

Torch is consistently the slowest system, and its MTBT grows the fastest. This is because, as an eager-execution system, Torch does not make use of compilation-based optimizations. For $T < 4,096$, TEMPO and JAX behave similarly, as both process the entire sequence as a single padded tensor. For $T \geq 4,096$, however, JAX continues to add padding, while TEMPO begins tiling attention (§4.3). This causes an initial slow-down at $T=4,096$ as TEMPO's tiling reduces parallelism, but a growing speedup for $T > 4,096$, due to reduced padding overhead. TEMPO is up to 43% faster at $T=16,384$, and while this gap should grow with T , all systems run out of GPU memory at $T=16,384$ due to K/V cache pressure.

Decoding long sequences. To scale to longer sequences, we lower the batch size to 4 and repeat the experiment. Fig. 17b shows the results. Similar behavior is observed, with 16,384 now serving as the point at which TEMPO's tiling reduces parallelism and JAX starts accumulating padding overhead. However, the longer sequence lengths cause more drastic slowdown for JAX, as the amount of padding it processes grows linearly with T . At $T=32,768$, TEMPO is 2× faster than JAX, and that grows to 2.5× faster at $T=65,536$.

Static tile size. These results can be better understood by studying how TEMPO's tiling strategy (§4.3) impacts decoding performance. The tile size used to decompose causal attention into statically-sized tiles is a key parameter that controls the trade-off between parallelism and padding overhead.

Fig. 18 shows the tile size affects latency for a fixed decode length of $T=16,384$ tokens and varying batch size B . At $B=4$, increasing the tile size up to $Z=8,192$ progressively lowers MTBT despite adding large padding, because the GPU has spare capacity to process this padding. At $BS=16$, however, MTBT decreases until the GPU utilization is maximized with $Z=2,048$, at which point it begins increasing again, yielding a *u-shaped* latency curve. This is because padding adds overhead when there is no spare GPU capacity. For medium to long sequence lengths, JAX is essentially in the low spare GPU capacity regime, and thus it is affected by padding overheads.

Windowed attention. Another method for scaling LLMs to longer sequences is to switch to a windowed attention pattern. Fig. 17c shows the results for Llama-3.2-3B with windowed attention at $B=16$. TEMPO and Torch achieve near constant MTBTs due to their support for dynamic shapes. Torch is eager however, while TEMPO enjoys whole-program compilation, making TEMPO up to 3.9× faster. JAX retains the exact same MTBT profile as for causal attention (Fig. 17a) since it models dynamic dependencies using masking over a spatial dimension. At $T=16,384$ TEMPO is up to 7× faster. Furthermore, TEMPO is the only system that adapts memory management to window dynamic dependencies. TEMPO uses a circular store for K/Vs and infers when old K/Vs can be deallocated, scaling to at least 4× longer sequences.

These conclusions are further confirmed by examining the runtime GPU memory usage (Fig. 19). JAX and Torch retain the same memory usage behavior between causal and windowed attention, i.e., they preallocate large buffers up-front and never deallocate memory. In contrast, TEMPO effectively manages memory according to algorithm-specific dynamic dependencies. For causal attention, TEMPO's memory usage grows only as TEMPO requires more static tiles to be processed, yielding a stepped usage pattern; for windowed attention, TEMPO's circular buffer tensor storage uses a small static allocation for all sequence lengths.

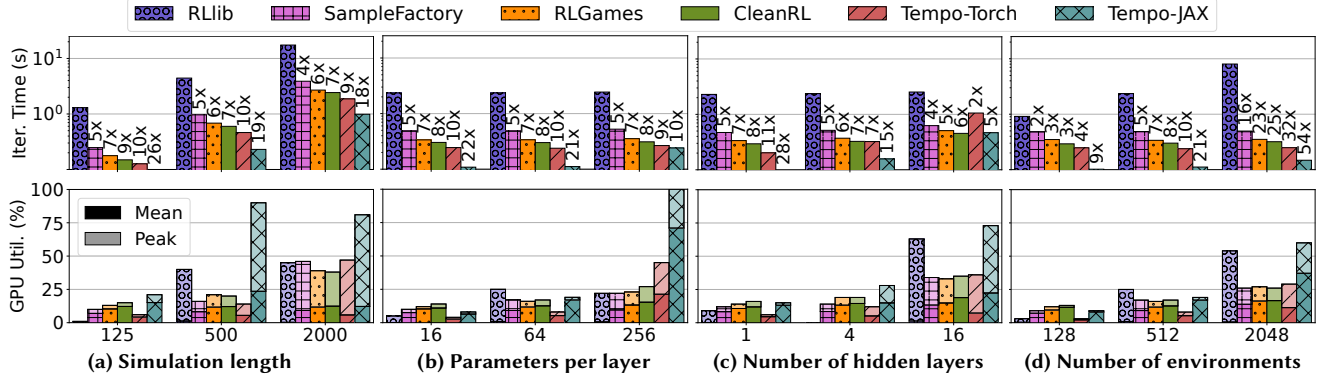


Fig. 20: End-to-end iteration time and GPU utilization for PPO at small-to-medium scale.

7.3 Training performance for RL

We evaluate the end-to-end RL training performance of TEMPO with PPO, a popular RL algorithm. PPO includes an anti-causal dependency (Fig. 2d) that prevents parallel acting and learning. We start with a default configuration with a simulation length of 250, 512 environments, 64 parameters per layer, 2 hidden layers, and a $3 \times 4 \times 4$ observation shape and then vary each parameter individually.

Small-to-medium scale PPO. Fig. 20 shows the result by reporting iteration time, GPU utilization, and speed-up factors over the slowest system. TEMPO has consistently lower iteration times than all other baselines: it is up to $54\times$ faster than RLib, and on average $2.6\times$ faster than the next fastest system, CleanRL, which is a hand-optimized Torch PPO implementation. This is due to TEMPO’s whole-program compilation: TEMPO deduplicates the forward passes of actor and learner by caching actor activations. In addition, TEMPO’s vectorization of the timestep dimension, efficient scheduling, and code-generation further improve performance.

As simulation length grows (Fig. 20a), all systems slow down linearly, narrowing relative performance differences. Adjusting the number of parameters per layer (Fig. 20b) does not affect relative performance until 256 parameters, when TEMPO begins tiling the backward pass. Increasing hidden layers (Fig. 20c) slows down all systems, but particularly TEMPO. This is likely due to Python-level AST interpretation overheads, which become apparent at this smaller scale and could be avoided by using an efficient C++ AST executor. We confirm this by the fact that adding environments (Fig. 20d) improves TEMPO’s relative performance, because AST interpretation overheads are amortized.

TEMPO’s Torch backend is consistently slower (by $2\times$ on average) than the JAX backend. This highlights the importance of pairing TEMPO with a capable code-generator such as XLA [36], with support for Triton [76] to find optimized kernels, CUDAGraphs [88] to lower dispatch overheads, and buffer donations (§5.2) to avoid allocations.

At this scale, the GPU is generally underutilized by all systems. However, TEMPO’s GPU utilization is on average

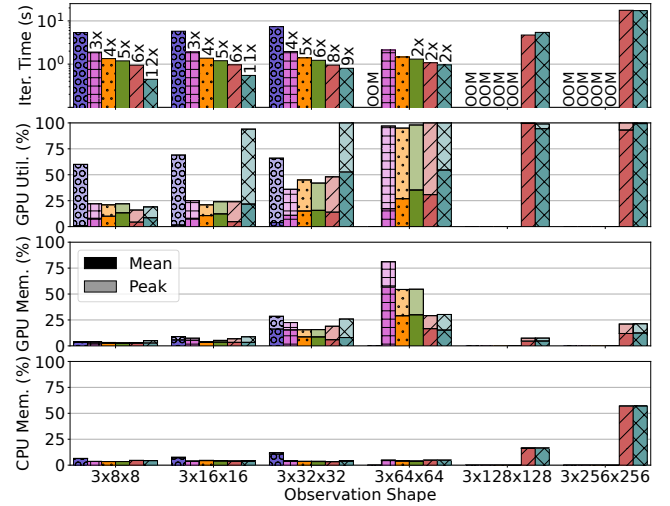


Fig. 21: Scaling to large observation sizes with PPO

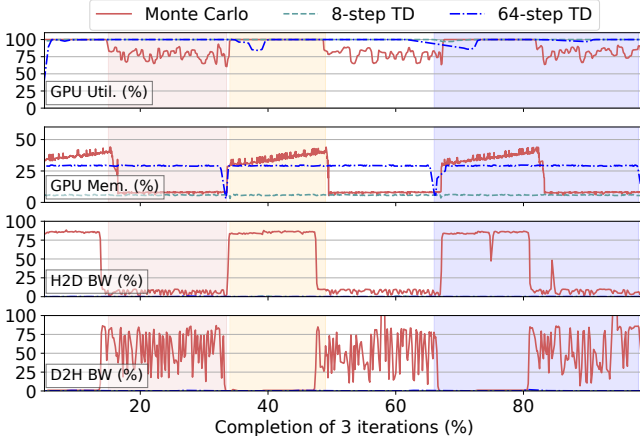
higher than the other systems, which indicates lower dispatch overheads due to our vectorization and dataflow fusion optimizations. Peak utilization is also high for RLib, but this is due to CPU/GPU data transfers performed every timestep.

Scaling to large observations. Training RL models from image observations is a common use case [89–91] that actor-learner RL frameworks struggle with. Higher image resolutions improve task performance, but require more memory. In this experiment, we halve the number of environments to 256, set the simulation length to 1,000, and show iteration times, GPU utilization and memory, and CPU memory, as we scale observations up to $3 \times 256 \times 256$.

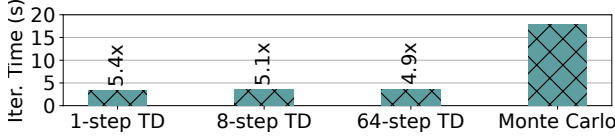
As Fig. 21 shows, the actor-learner approach stores and learns from all observations at once, resulting in a GPU peak memory usage that exceeds capacity. RLib enters a fail-retry loop before $3 \times 64 \times 64$, while the other baselines fail to scale due to out-of-memory (OOM) errors. In contrast, TEMPO scales to $3 \times 256 \times 256$ ($16\times$ larger) by balancing GPU/CPU memory usage through tiling and swapping (see §4.3 and §5.2), without the need for a replay buffer. TEMPO uses tiling after $3 \times 64 \times 64$, leading to a slight slow-down, and swapping is used after $3 \times 128 \times 128$. To scale PPO further, TEMPO would only require adding more CPU memory.



Fig. 22: Activation caching without symbolic differentiation



(a) Aligned runtime metrics for different algorithms



(b) Iteration times for REINFORCE variations

Fig. 23: Algorithm-specific scheduling with TEMPO-JAX

Activation caching. We modify CleanRL to also cache and reuse forward pass activations during learning. Fig. 22 shows that CleanRL’s iteration time is on average 2× slower with caching (CleanRL (C)). In Torch, which CleanRL uses, each forward pass creates a corresponding backpropagation graph. The full backward graph concatenates each timestep’s graph, causing *graph size explosion*. Thus, Torch must process thousands of small subgraphs, incurring large overhead. In TEMPO, forward pass activations have an explicit timestep dimension, which is automatically vectorized (§4.2), enabling an efficient backpropagation graph.

7.4 Algorithm-specific scheduling

We explore TEMPO’s ability to adapt scheduling to algorithm-specific dynamic dependencies in the large observation setting ($3 \times 256 \times 256$). We use the two variations of REINFORCE shown in Alg. 1, which use: (i) traditional Monte Carlo [15] returns; and (ii) n -step temporal-difference (TD) returns [14].

Fig. 23a shows how TEMPO executes these two algorithms. Since Monte Carlo (red line) uses an anti-causal access pattern ($r[t : T]$), TEMPO’s scheduler waits until the simulation finishes before learning. During simulation (red shading), TEMPO continually evicts observations to CPU memory, maintaining low GPU memory usage, but not fully utilizing the GPU. Once finished, learning begins (orange shading),

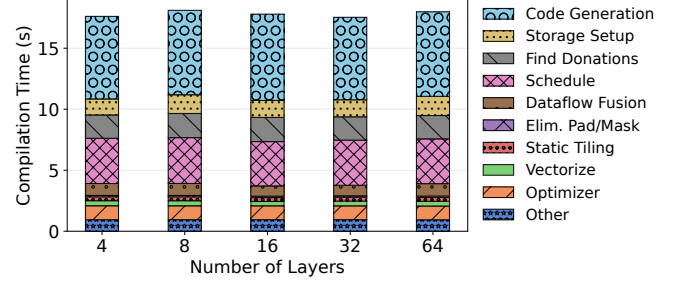


Fig. 24: Compilation breakdown for different model sizes

and TEMPO incrementally swaps observations back to GPU memory, learning from them while fully utilizing the GPU.

With n -step TD (blue and teal lines), however, TEMPO’s scheduler finds a different strategy: due to the forward window access pattern ($r[t : \min(t+n, T)]$), after an n -step delay, it begins computing gradients *in parallel* with simulation (blue shading). This enables TEMPO to: (i) fully utilize the GPU; (ii) only store the last n observations, deallocating older data as early as possible; and (iii) avoid swapping due to the lower memory demand. Due to the 64-step delay before learning starts, 64-step TD still experiences a slight GPU utilization dip, which 8-step TD does not.

For smaller n , TEMPO parallelizes acting and learning sooner, leading to faster iteration times, as shown in Fig. 23b. Note that actor-learner frameworks (see §2.2) cannot use such parallel strategies due to coarsening dynamic dependencies between acting and learning to $[0 : T]$. For either algorithm, TEMPO finds an effective schedule, allowing for large-scale training on a single GPU.

7.5 Compilation scalability

We evaluate the scalability of TEMPO’s compilation process by varying the number of transformer blocks in the Llama-3.2 architecture. Fig. 24 shows that the compilation time remains near constant as the model size is increased. This is due to TEMPO’s ability to exploit repeated structure in models with a temporal dimension l for layers, creating small SDGs even for large models. Each layer l depends upon the previous layer $l-1$, and the final output is given by layer $L-1$.

Compilation times are around 18 s, of which 7 s are spent in DL backend code-generation and 1 s is spent on initial allocations for tensor stores. Beyond this, the majority of the compilation time is taken by polyhedral scheduling and buffer donation analysis. This is because these steps require invocations of the polyhedral library [34], which parses constraint strings. Due to TEMPO’s fusion and layer encoding, ILP solving accounts for only 2 s of the scheduling time. The remaining compiler passes are fast, because they involve only SDG and symbolic manipulations. Vectorization, tiling and the optimizer each take under 1 s to complete, despite their fairly complex transformations. Dataflow fusion takes slightly longer, because it performs a large number of validity checks for each fusion.

8 Discussion

Distribution. While TEMPO is currently a single-GPU system, its temporal tensor dimensions can serve as a foundation for distribution [92]. This requires identifying temporal dimensions that can be mapped to parallel for-loops in the schedule. TEMPO can create such temporal dimensions by tiling (§4.3) the SDG across a new “worker” dimension. If the starting point is an input-data tensor, this gives *data-parallelism* [93]; in combination with SDG cuts (partitioning the SDG into per-worker subgraphs), input-data tiling also yields *pipeline parallelism* [75]; if tiling starts from a weight tensor, we obtain *model-parallelism* [94]. TEMPO can then assign schedule partitions to workers, which perform reductions as collective communication operations.

Dynamic termination. TEMPO does not yet support dynamic termination of temporal dimensions (e.g., T determined at runtime). Prior work [77] has shown that this can be achieved by generating schedules that break out of infinite loops when the termination condition is met. Evaluating termination at each step, however, is expensive, because it requires a GPU/CPU transfer. We plan to explore speculative methods to reduce this overhead in future work.

Smart transformation policies. While TEMPO proposes powerful transformations, it often lacks sophisticated policies for applying them. For example, TEMPO’s swapping augmentations are oblivious to kernel execution latencies, causing stalls when dependent operations wait for data. This can be addressed through adding constraints to the scheduling problem based on kernel profiling. Additionally, tiling is currently triggered based on operator’s point memory usage, and not on a more accurate memory usage analysis of the whole program. Finally, the tile size used in static tiling is currently set by the user, which can be suboptimal. The ideal tile size, typically the smallest size that maximizes GPU utilization (§7.2), could instead be found through binary search.

Optimized kernels. LLM programs today use hand-crafted, high-performance kernels, such as FlashAttention [95], which are not currently supported by TEMPO. Adding these as first-class operators to TEMPO is difficult due to their complex semantics [96], which prevents TEMPO from applying its transformations (vectorization, tiling, etc.). One approach is to introduce a final compiler pass, before scheduling, that identifies attention-like patterns in the SDG and lifts them (§4.1) into a single FlashAttention kernel operator.

9 Related Work

Memory management in DL. Prior work has explored gradient accumulation [97] and swapping [20, 98] to reduce memory pressure. TEMPO transparently integrates these ideas at a program level through transformations (tiling) and scheduling (loads and evicts). Recomputation [80, 81] of intermediate activations can also reduce peak memory usage. TEMPO does not currently offer automatic recomputation.

Polyhedral model in DL. Tensor comprehensions [28] and Tiramisu [99] optimize tensor computations through polyhedral loop fusions, tilings, and parallelization. PPCG [100] is a C-to-CUDA compiler that performs similar loop optimizations and manages memory across the GPU hierarchy. In general, these approaches use the polyhedral model for kernel-level optimizations, while TEMPO employs it for whole-program execution scheduling and memory management.

DL compilers. TVM [25] and Triton [76] are kernel-level compilers but do not use the polyhedral model. Each introduces a domain-specific language for expressing and optimizing low-level tensor computations. MLIR [27] is a more general compiler infrastructure, which can be used to build domain-specific compilers, including for DL [36]. TEMPO can leverage these systems for kernel code-generation.

Support for dynamic tensor shapes. JANUS [101] and PyTorch 2 [16] have some support for dynamic DL compilation through a combination of symbolic tensor shapes, speculative compilation, and assertions that trigger recompilation. These approaches, however, suffer from recompilation overheads when shapes change. Relax [17] targets compilation for model inference but requires user annotations for dynamic shapes. Nimble [102] and DISC [29] support dynamic tensor shapes through runtime adaptation. Nimble uses an interpreted approach, while DISC compiles just-in-time. While prior approaches support dynamic shapes, TEMPO is the first system to focus on dynamic dependencies, offering a complete solution with symbolic automatic differentiation, and ahead-of-time whole-program optimization, scheduling, and memory planning with dynamic dependencies.

10 Conclusion

We described TEMPO, a system that combines the flexibility of eager execution with the program-wide optimizations and scheduling of graph-based execution for dynamic DL algorithms. TEMPO introduces *recurrent tensors*, which add explicit *temporal dimensions* to tensors that can be indexed using *symbolic expressions* to express dynamic dependencies. TEMPO uses *symbolic dependence graphs* (SDG), to extend traditional optimizations to dynamic DL programs: it views vectorization, and tiling as transformations that move tensor dimensions from temporal to spatial and vice versa. TEMPO leverages tiling to decompose dynamic computations into static-size tiles, which enables code-generation with minimal padding. To find valid execution schedules, TEMPO uses the polyhedral model and augments SDGs to schedule memory management operations. TEMPO achieves substantial performance speed-ups, which demonstrates its potential of whole-program optimization for dynamic DL workloads.

Acknowledgements. We thank our anonymous reviewers and our shepherd for their insightful feedback and suggestions, which greatly improved this paper.

References

- [1] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladen Koltun, and Marco Hutter. Learning Agile and Dynamic Motor Skills for Legged Robots. *Science Robotics*, 2019.
- [2] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the Game of Go Without Human Knowledge. *Nature*, 2017.
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, et al. Highly Accurate Protein Structure Prediction with AlphaFold. *Nature*, 2021.
- [5] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [6] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [7] Roy Frostig, Matthew James Johnson, and Chris Leary. Compiling Machine Learning Programs via High-Level Tracing. In *Conference on Machine Learning and Systems (MLSys)*, 2018.
- [8] Rajat Raina, Anand Madhavan, Andrew Y Ng, et al. Large-Scale Deep Unsupervised Learning Using Graphics Processors. In *International Conference on Machine Learning (ICML)*, 2009.
- [9] A Vaswani. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [10] Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT press, 2018.
- [11] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning Representations by Back-Propagating Errors. *Nature*, 1986.
- [12] Albert Gu, Karan Goel, and Christopher Ré. Efficiently Modeling Long Sequences with Structured State Spaces. *arXiv preprint arXiv:2111.00396*, 2021.
- [13] Richard M Karp, Raymond E Miller, and Shmuel Winograd. The Organization of Computations for Uniform Recurrence Equations. *Journal of the ACM (JACM)*, 1967.
- [14] Richard S Sutton. Learning to Predict by the Methods of Temporal Differences. *Machine Learning (Mach. Learn.)*, 1988.
- [15] Ronald J Williams. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine learning (Mach. Learn.)*, 1992.
- [16] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [17] Ruihang Lai, Junru Shao, Siyuan Feng, Steven Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, Lesheng Jin, Yaxing Cai, Ziheng Jiang, Yong Wu, Sunghyun Park, Prakash Srivastava, Jared Roesch, Todd C. Mowry, and Tianqi Chen. Relax: Composable Abstractions for End-to-End Dynamic Machine Learning. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.
- [18] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. TensorFlow: A System for Large-Scale Machine Learning. In *Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.
- [19] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elilol, Zongheng Yang, William Paul, Michael I Jordan, et al. Ray: A Distributed Framework for Emerging AI Applications. In *Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.
- [20] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W Keckler. vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design. In *International Symposium on Microarchitecture (MICRO)*, 2016.
- [21] Akshay Agrawal, Akshay Modi, Alexandre Passos, Allen Lavoie, Ashish Agarwal, Asim Shankar, Igor Ganchev, Josh Levenberg, Ming-sheng Hong, Rajat Monga, et al. TensorFlow Eager: A Multi-Stage, Python-Embedded DSL for Machine Learning. In *Conference on Machine Learning and Systems (MLSys)*, 2019.
- [22] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. CuDNN: Efficient Primitives for Deep Learning. *arXiv preprint arXiv:1410.0759*, 2014.
- [23] Martin Maas, Ulysse Beaunon, Arun Chauhan, and Berkin Ilbeyi. Telamalloc: Efficient On-Chip Memory Allocation for Production Machine Learning Accelerators. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2022.
- [24] Siran Liu, Chengxiang Qi, Ying Cao, Chao Yang, Weifang Hu, Xuanhua Shi, Fan Yang, and Mao Yang. Uncovering Nested Data Parallelism and Data Reuse in DNN Computation with FractalTensor. In *Symposium on Operating Systems Principles (SOSP)*, 2024.
- [25] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.
- [26] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. *ACM SIGPLAN Notices*, 2013.
- [27] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *International Symposium on Code Generation and Optimization (CGO)*, 2021.
- [28] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. *arXiv preprint arXiv:1802.04730*, 2018.
- [29] Zhen Zheng, Zaifeng Pan, Dalin Wang, Kai Zhu, Wenyi Zhao, Tianyou Guo, Xiafei Qiu, Minmin Sun, Junjie Bai, Feng Zhang, et al. Bladedisc: Optimizing Dynamic Shape Machine Learning Workloads via Compiler Approach. In *ACM Conference on Management of Data (SIGMOD)*, 2023.
- [30] Matteo Hessel, Manuel Kroiss, Aidan Clark, Iurii Kemaev, John Quan, Thomas Keck, Fabio Viola, and Hado van Hasselt. Podracer Architectures for Scalable Reinforcement Learning. *arXiv preprint arXiv:2104.06272*, 2021.
- [31] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [32] Stefan Van Der Walt, S Chris Colbert, and Gael Varoquaux. The NumPy Array: A Structure for Efficient Numerical Computation. *Computing in Science & Engineering (Comput. Sci. Eng.)*, 2011.

- [33] Atılım Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic Differentiation in Machine Learning: A Survey. *Journal of Machine Learning Research (JMLR)*, 2018.
- [34] Sven Verdoolaege. isl: An Integer Set Library for the Polyhedral Model. In *International Congress on Mathematical Software (ICMS)*, 2010.
- [35] Uday Bondhugula, Albert Hartono, J Ramanujam, and P Sadayappan. Pluto: A Practical and Fully Automatic Polyhedral Program Optimization System. In *Conference on Programming Language Design and Implementation (PLDI)*, 2008.
- [36] OpenXLA Team. XLA (Accelerated Linear Algebra), 2024.
- [37] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*, 2024.
- [38] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [39] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [40] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [41] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous Methods for Deep Reinforcement Learning. In *International Conference on Machine Learning (ICML)*, 2016.
- [42] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big Bird: Transformers for Longer Sequences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [43] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [44] Colin Lea, Michael D Flynn, Rene Vidal, Austin Reiter, and Gregory D Hager. Temporal Convolutional Networks for Action Segmentation and Detection. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [45] Xuezhe Ma, Chunting Zhou, Xiang Kong, Junxian He, Liangke Gui, Graham Neubig, Jonathan May, and Luke Zettlemoyer. Mega: Moving Average Equipped Gated Attention. *arXiv preprint arXiv:2209.10655*, 2022.
- [46] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. *arXiv preprint arXiv:1901.02860*, 2019.
- [47] Alex Suhan, Davide Libenzi, Ailing Zhang, Parker Schuh, Brennan Saeta, Jie Young Sohn, and Denys Shabalov. LazyTensor: Combining Eager Execution with Domain-Specific Compilers. *arXiv preprint arXiv:2102.13267*, 2021.
- [48] Yuan Yu, Martín Abadi, Paul Barham, Eugene Brevdo, Mike Burrows, Andy Davis, Jeff Dean, Sanjay Ghemawat, Tim Harley, Peter Hawkins, et al. Dynamic Control Flow in Large-Scale Machine Learning. In *European Conference on Computer Systems (EuroSys)*, 2018.
- [49] OpenXLA Team. Operation Semantics | XLA, 2025. OpenXLA Team.
- [50] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*, 2023.
- [51] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-Centric Disaggregated Architecture for LLM Serving. *arXiv preprint arXiv:2407.00079*, 2024.
- [52] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers Are RNNs: Fast Autoregressive Transformers with Linear Attention. In *International Conference on Machine Learning (ICML)*, 2020.
- [53] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019.
- [54] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving Open Language Models at a Practical Size. *arXiv preprint arXiv:2408.00118*, 2024.
- [55] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [56] Adam Stooke and Pieter Abbeel. Rlpyt: A Research Code Base for Deep Reinforcement Learning in PyTorch. *arXiv preprint arXiv:1909.01500*, 2019.
- [57] Aleksei Petrenko, Zhehui Huang, T. Kumar, G. Sukhatme, and V. Koltun. Sample Factory: Egocentric 3D Control from Pixels at 100000 FPS with Asynchronous Reinforcement Learning. *arXiv preprint arXiv:2006.11751*, 2020.
- [58] Matt Hoffman, Bobak Shahriari, John Aslanides, Gabriel Barth-Maron, Feryal Behbahani, Tamara Norman, Abbas Abdolmaleki, Albin Cassirer, Fan Yang, Kate Baumli, et al. Acme: A Research Framework for Distributed Reinforcement Learning. *arXiv preprint arXiv:2006.00979*, 2020.
- [59] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan, and Ion Stoica. RLlib: Abstractions for Distributed Reinforcement Learning. In *International Conference on Machine Learning (ICML)*, 2018.
- [60] Jiayi Weng, Huayu Chen, Dong Yan, Kaichao You, Alexis Duburcq, Minghao Zhang, Hang Su, and Jun Zhu. Tianshou: A Highly Modularized Deep Reinforcement Learning Library. *arXiv preprint arXiv:2107.14171*, 2021.
- [61] Denys Makoviichuk and Viktor Makoviychuk. rl-games: A High-performance Framework for Reinforcement Learning, 2022.
- [62] Albert Bou, Matteo Bettini, Sebastian Dittert, Vikash Kumar, Shagun Sodhani, Xiaomeng Yang, Gianni De Fabritiis, and Vincent Moens. TorchRL: A Data-Driven Decision-Making Library for PyTorch. *arXiv preprint arXiv:2306.00577*, 2023.
- [63] Albin Cassirer, Gabriel Barth-Maron, Eugene Brevdo, Sabela Ramos, Toby Boyd, Thibault Sottiaux, and Manuel Kroiss. Reverb: A Framework for Experience Replay. *arXiv preprint arXiv:2102.04736*, 2021.
- [64] Hanjing Wang, Man-Kit Sit, Congjie He, Ying Wen, Weinan Zhang, Jun Wang, Yaodong Yang, and Luo Mai. GEAR: A GPU-Centric Experience Replay System for Large Reinforcement Learning Models. In *International Conference on Machine Learning (ICML)*, 2023.
- [65] Diederik P Kingma. Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [66] Christian Darken and John Moody. Note on Learning Rate Schedules for Stochastic Optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1990.
- [67] Paul J Werbos. Backpropagation through Time: What It Does and How to Do It. *Proceedings of the IEEE*, 1990.

- [68] Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow Network Based Generative Models for Non-Iterative Diverse Candidate Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [69] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems. *arXiv preprint arXiv:2005.01643*, 2020.
- [70] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [71] Taku Kudo and John Richardson. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. *arXiv preprint arXiv:1808.06226*, 2018.
- [72] Ken Kennedy and John R Allen. *Optimizing Compilers for Modern Architectures: A Dependence-Based Approach*. Morgan Kaufmann Publishers Inc., 2001.
- [73] Guy E Blelloch. Prefix Sums and their Applications. 1990.
- [74] Michael J Flynn. Very High-Speed Computing Systems. *Proceedings of the IEEE*, 2005.
- [75] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [76] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.
- [77] Mohamed-Walid Benabderrahmane, Louis-Noël Pouchet, Albert Cohen, and Cédric Bastoul. The Polyhedral Model is More Widely Applicable than You Think. In *International Conference on Compiler Construction (CC)*, 2010.
- [78] Tobias Grosser, Armin Groesslinger, and Christian Lengauer. Polly: Performing Polyhedral Optimizations on a Low-Level Intermediate Representation. *Parallel Processing Letters*, 2012.
- [79] Sven Verdoolaege and Tobias Grosser. Polyhedral Extraction Tool. In *International Workshop on Polyhedral Compilation Techniques (IMPACT)*, 2012.
- [80] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. Capuchin: Tensor-based GPU Memory Management for Deep Learning. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.
- [81] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. SuperNeurons: Dynamic GPU Memory Management for Training Deep Neural Networks. In *Symposium on Parallelism in Algorithms and Architectures (SPAA)*, 2018.
- [82] Peter J Denning. The Working Set Model for Program Behavior. *Communications of the ACM (CACM)*, 1968.
- [83] Tobias Grosser, Sven Verdoolaege, and Albert Cohen. Polyhedral AST Generation is More than Scanning Polyhedra. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 2015.
- [84] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [85] Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and JoÅGo GM AraÅsjo. CleanRL: High-Quality Single-File Implementations of Deep Reinforcement Learning Algorithms. *Journal of Machine Learning Research (JMLR)*, 2022.
- [86] C Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. Brax—A Differentiable Physics Engine for Large Scale Rigid Body Simulation. *arXiv preprint arXiv:2106.13281*, 2021.
- [87] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, et al. Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning. *arXiv preprint arXiv:2108.10470*, 2021.
- [88] NVIDIA Corporation. Getting Started with CUDA Graphs, 2020.
- [89] Ankur Handa, Arthur Allshire, Viktor Makoviychuk, Aleksei Petrenko, Ritvik Singh, Jingzhou Liu, Denys Makoviychuk, Karl Van Wyk, Alexander Zhurkevich, Balakumar Sundaralingam, et al. Dextreme: Transfer of Agile In-Hand Manipulation from Simulation to Reality. In *International Conference on Robotics and Automation (ICRA)*, 2023.
- [90] B Ravi Kiran, Ibrahim Sobh, Victor Talpaert, Patrick Mannion, Ahmad A Al Sallab, Senthil Yogamani, and Patrick Pérez. Deep Reinforcement Learning for Autonomous Driving: A Survey. *IEEE Transactions on Intelligent Transportation Systems (T-ITS)*, 2021.
- [91] Chao Yu, Jiming Liu, Shamim Nemati, and Guosheng Yin. Reinforcement Learning in Healthcare: A Survey. *ACM Computing Surveys (CSUR)*, 2021.
- [92] Pedro F. Silvestre and Peter Pietzuch. Systems Opportunities for LLM Fine-Tuning using Reinforcement Learning. In *Workshop on Machine Learning and Systems (EuroMLSys)*, 2025.
- [93] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. ImageNet Classification with Deep Convolutional Neural Networks. *Communications of the ACM (CACM)*, 2017.
- [94] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [95] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [96] Paul Barham and Michael Isard. Machine Learning Systems Are Stuck in a Rut. In *Workshop on Hot Topics in Operating Systems (HotOS)*, 2019.
- [97] Shixiong Zhao, Fanxin Li, Xusheng Chen, Xiuxian Guan, Jianyu Jiang, Dong Huang, Yuhao Qing, Sen Wang, Peng Wang, Gong Zhang, et al. vPipe: A Virtualized Acceleration System for Achieving Efficient and Scalable Pipeline Parallel DNN Training. *Transactions on Parallel and Distributed Systems (TPDS)*, 2021.
- [98] Chien-Chin Huang, Gu Jin, and Jinyang Li. Swapadvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.
- [99] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman Amarasinghe. Tiramisu: A Polyhedral Compiler for Expressing Fast and Portable Code. In *International Symposium on Code Generation and Optimization (CGO)*, 2019.
- [100] Sven Verdoolaege, Juan Carlos Juega, Albert Cohen, Jose Ignacio Gomez, Christian Tenllado, and Francky Catthoor. Polyhedral Parallel Code Generation for CUDA. *ACM Transactions on Architecture and Code Optimization (TACO)*, 2013.
- [101] Eunji Jeong, Sungwoo Cho, Gyeong-In Yu, Joo Seong Jeong, Dong-Jin Shin, Taebum Kim, and Byung-Gon Chun. Speculative Symbolic Graph Execution of Imperative Deep Learning Programs. *ACM SIGOPS Operating Systems Review*, 2019.
- [102] Haichen Shen, Jared Roesch, Zhi Chen, Wei Chen, Yong Wu, Mu Li, Vin Sharma, Zachary Tatlock, and Yida Wang. Nimble: Efficiently Compiling Dynamic Neural Networks for Model Inference. In *Conference on Machine Learning and Systems (MLSys)*, 2021.